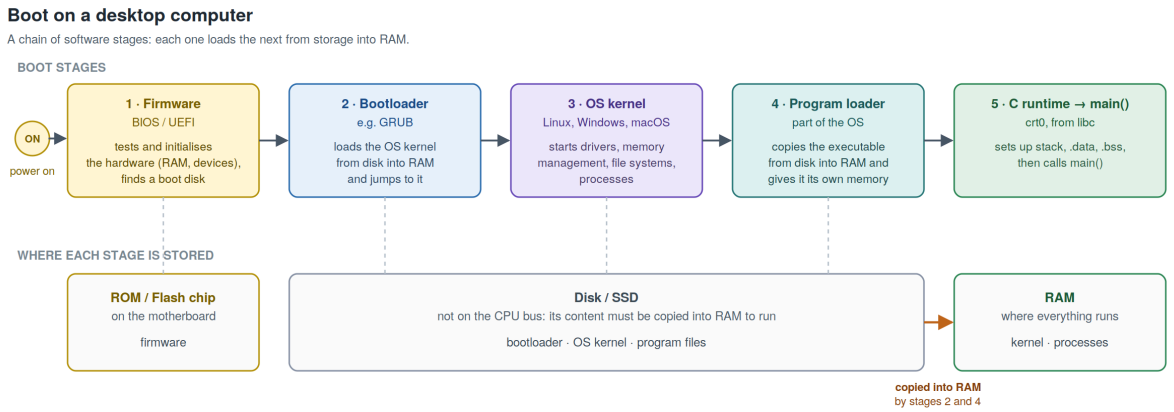
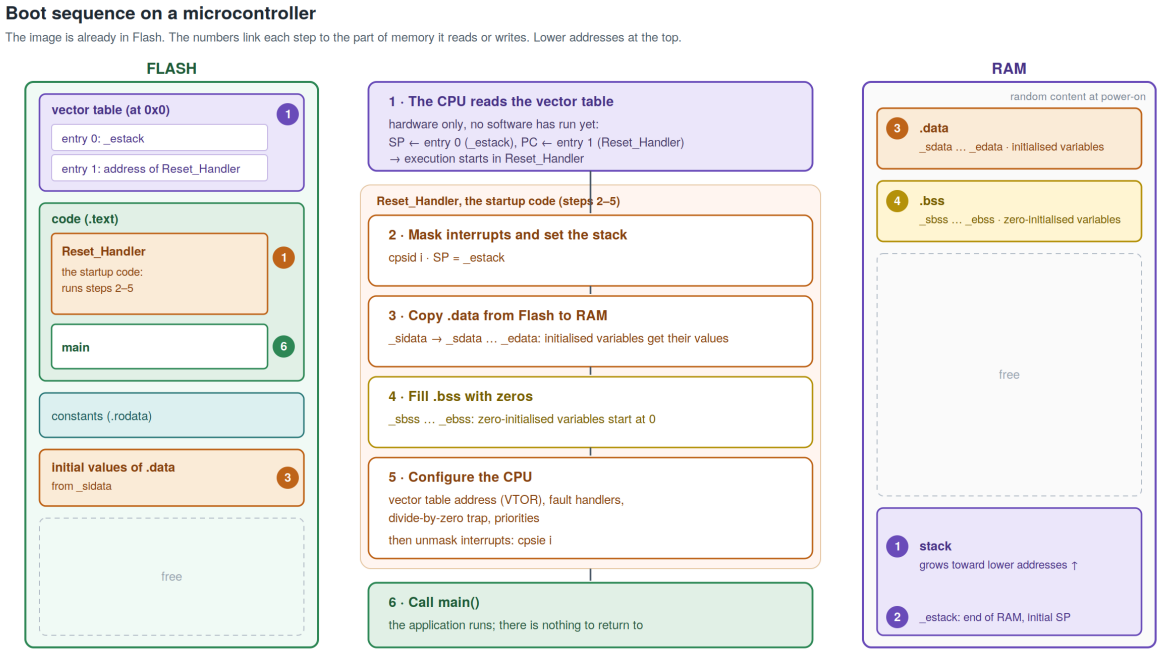
1 The Boot Problem

On a desktop, the operating system prepares everything before a program runs: its loader copies the executable from disk into RAM, sets up memory, and calls `main()`.



By the time main() runs, firmware, bootloader, kernel and loader have already prepared the hardware and the memory.

A microcontroller has no operating system and no loader. At reset the CPU only reads two words at address `0x0`, the initial stack pointer and the address of the first instruction, and starts executing. The code runs directly from Flash, but RAM holds random values at power-on: before `main()`, the program itself must set up the stack, copy the initial values of the variables from Flash to RAM, and clear the zero-initialised ones.



Order matters: the CPU must know where to start (step 1) before any instruction, including the copy of .data, can run.

The job is split between the **linker script**, used at build time, and the **startup code**, run right after reset.

2 Linker Script

A **linker script** is a text file, passed to the linker with `-T`, that describes the memories of the chip and how the program must be placed in them. The linker alone does not know which memories exist or where they are: the

script tells it, so that it can assign every address, place each section in Flash or RAM, define the symbols the startup code needs, and fail the link if something does not fit.

```
MEMORY
{
    FLASH (rx)  : ORIGIN = 0x00000000, LENGTH = 4M
    RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 4M
}

ENTRY(Reset_Handler)

_estack = ORIGIN(RAM) + LENGTH(RAM);
_Min_Stack_Size = 0x1000;
_Min_Heap_Size = 0x1000;

SECTIONS
{
    .isr_vector :
    {
        . = ALIGN(4);
        KEEP(*(.isr_vector))
        . = ALIGN(4);
    } > FLASH

    .text :
    {
        . = ALIGN(4);
        *(.text*)
        *(.rodata*)
        . = ALIGN(4);
        _etext = .;
    } > FLASH

    _sidata = LOADADDR(.data);

    .data :
    {
        . = ALIGN(4);
        _sdata = .;
        *(.data*)
        . = ALIGN(4);
        _edata = .;
    } > RAM AT > FLASH
```

```
    .bss :
    {
        . = ALIGN(4);
        _sbss = .;
        *(.bss*)
        *(COMMON)
        . = ALIGN(4);
        _ebss = .;
    } > RAM

    .heap :
    {
        . = ALIGN(8);
        PROVIDE( end = . );
        PROVIDE( _end = . );
        _heap_bottom = .;
        . = . + _Min_Heap_Size;
        _heap_top = .;
    } > RAM

    __StackLimit = _estack - _Min_Stack_Size;
    ASSERT(__StackLimit >= _heap_top, "region RAM overflowed: heap collides with the stack")
}
```

Part	Role
`MEMORY { ... }`	declares the available memory regions: name, start address (`ORIGIN`), size (`LENGTH`), attributes (`r` , `w` , `x`)
`SECTIONS { ... }`	builds the output sections from the input sections of the object files and assigns each one to a memory region
`ENTRY(...)`	names the symbol where execution starts; used by tools (debugger, QEMU), not by the CPU
Symbol assignments (`_estack = ...;`)	create named addresses that the code can use, such as the top of the stack or the bounds of `.data` and `.bss`

2.1 MEMORY: Regions and Size Checking

Each line of `MEMORY` defines one **memory region**; the two regions correspond to the two populated memories of the Cortex-M3 memory map.

Field	Meaning	Example
Name	freely chosen identifier, not a keyword	`FLASH` , `RAM`

Field	Meaning	Example
Attributes	allowed access: `r` read, `w` write, `x` execute	`(rx)`, `(rwx)`
`ORIGIN`	start address of the region	`0x00000000`, `0x20000000`
`LENGTH`	size of the region	`4M`

The linker uses `LENGTH` to verify that the program fits: if the content assigned to a region does not fit, the link fails (e.g. `region 'FLASH' overflowed by 28 bytes`) instead of producing an image that cannot work.

The script also defines the symbol `_estack`, the **initial value** of the **stack pointer**: `ORIGIN(RAM) + LENGTH(RAM)` = `0x20400000`, just past the end of RAM. The stack grows **downwards**, so it starts at the top of RAM to have the most room. This address is never used itself: each push first decrements the stack pointer and then writes, so the first push writes at `0x203FFFFC`, the last word of RAM.

2.2 SECTIONS: From Input Sections to Output Sections

Each object file (`.o`) contains its own sections, called **input sections**:

- `.text` (code),
- `.rodata` (constants),
- `.data` (initialised variables),
- `.bss` (zero-initialised variables).

The linker merges them into the sections of the final ELF file, called **output sections**. The `SECTIONS` block contains one definition per output section, always with the same format:

```
<output_section> :
{
    <rule>            /* which input sections to take, e.g. *(.text*) */
    <rule>            /* rules are applied in order                   */
    <symbol> = .;     /* optional: give a name to the current address */
} > <REGION>          /* memory region where the section is placed    */
```

An **output section** is created with a name and placed in a memory region:

- `.text : { … }`: creates an output section called `.text` in the final ELF.
- `> FLASH`: places that output section in the FLASH region.

The rules inside the braces select the **input sections** that go into it:

- `*(.text*)`: from every object file (`*`), takes every input section whose name starts with `.text`.
- `*(.rodata*)`: same, for the read-only data.
- `KEEP(…)`: always keeps the selected input sections, even when the linker removes unused ones (garbage collection).

Addresses inside the section are controlled through the location counter:

- `.`: the **location counter**, the address where the linker is currently placing content.
- `. = ALIGN(4)`: rounds the location counter up to a multiple of 4.

- `_etext = .;`: defines a symbol equal to the current address, here the end of the code.

In this linker script the **mapping** is:

Input section (in each `.o`)	Created by	Rule	Output section	Region
`.isr_vector`	the startup code, on purpose	`KEEP(* (.isr_vector))`	`.isr_vector`	FLASH, at `0x0`
`.text`	the compiler: code	`*(.text*)`	`.text`	FLASH
`.rodata`	the compiler: constants	`*(.rodata*)`	`.text`	FLASH
`.data`	the compiler: initialised variables	`*(.data*)`	`.data`	RAM, stored in FLASH
`.bss`	the compiler: zero-initialised variables	`*(.bss*)`, `* (COMMON)`	`.bss`	RAM

The compiler chooses the names `.text`, `.rodata`, `.data`, and `.bss` by itself. `.isr_vector` is a name chosen by the startup code, which puts the vector table in a section of its own so that the linker script can place it first. `COMMON` holds globals declared without a value (`int counter;`) that some compilers keep in a separate pool; they belong in `.bss` too.

> **THE ORDER IN THE SCRIPT IS THE ORDER IN MEMORY**
>
> The linker places output sections in the order they are written in the script, and inside each output section it places input sections in the order of the rules. Because `.isr_vector` is the first output section in FLASH, the vector table starts at `0x0`, where the CPU reads it at reset. For the same reason, inside `.text` all the code comes first and all the constants after it.

2.2.1 Run Address (VMA) and Load Address (LMA)

```
_sidata = LOADADDR(.data);
.data :
{
    _sdata = .;       /* start of .data in RAM */
    *(.data*)
    _edata = .;       /* end of .data in RAM   */
} > RAM AT > FLASH
```

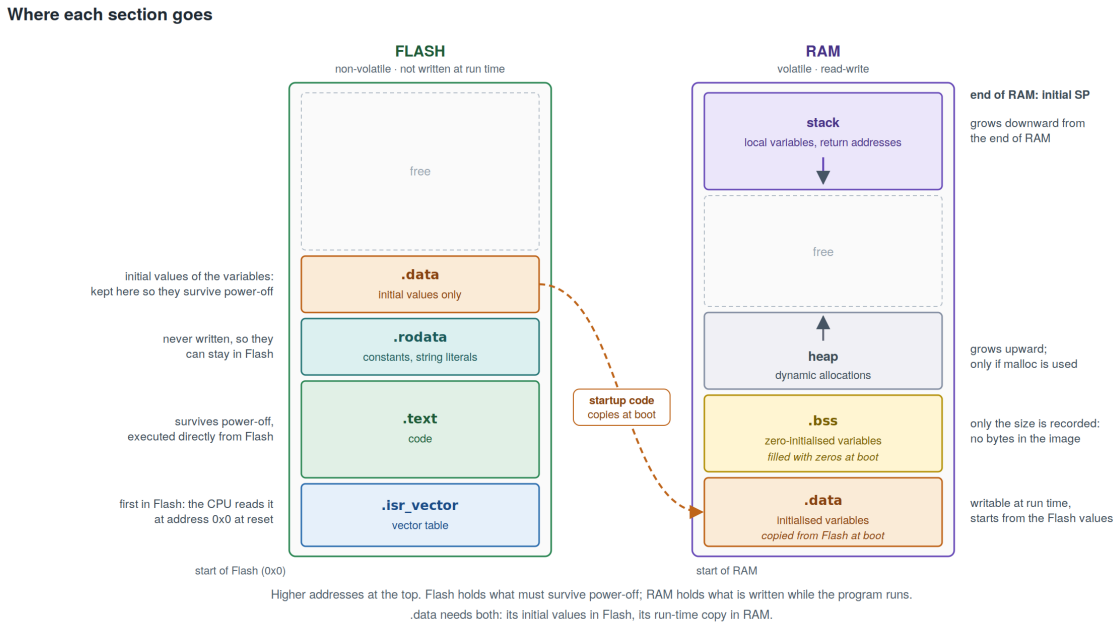
Every output section has two addresses:

Address	Meaning	In the script
VMA (virtual memory address, *run address*)	where the program uses the section at run time	`> REGION`
LMA (load memory address)	where the section's bytes are stored in the image	`AT > REGION` (if omitted, LMA = VMA)

For most sections the two coincide. `.data` is the exception: `> RAM AT > FLASH` gives it a VMA in RAM, where the variables are modified, and an LMA in FLASH, where their initial values survive power-off. The script also records three addresses for the boot code:

Symbol	Value	Role in the copy
`_sidata`	`LOADADDR(.data)`: start of the initial values in FLASH (LMA)	source
`_sdata`	start of `.data` in RAM (VMA)	destination start
`_edata`	end of `.data` in RAM (VMA)	destination end

> **NOTHING COPIES LMA TO VMA AUTOMATICALLY**
>
> The linker only records the two addresses, and the CPU copies nothing at reset. The startup code must copy `.data` from `_sidata` to `_sdata` … `_edata`.



2.2.2 Heap and Stack Reservation

The linker script also reserves space in RAM for the heap and the stack, and checks at link time that they do not overlap:

```
.heap :
{
    . = ALIGN(8);
    _heap_bottom = .;
    . = . + _Min_Heap_Size;
    _heap_top = .;
} > RAM


__StackLimit = _estack - _Min_Stack_Size;
```

```
ASSERT(__StackLimit >= _heap_top, "region RAM overflowed: heap collides with the stack")
```

- `. = . + _Min_Heap_Size`: advances the location counter by 4 KB, so nothing else can be placed in the heap.
- `__StackLimit` : `_estack - _Min_Stack_Size`, the lowest address the stack may reach; the stack is not a section, only this address range.
- `ASSERT(cond, msg)`: makes the link fail with `msg` if `cond` is false.

> **HEAP–STACK COLLISION CAUGHT AT LINK TIME**
>
> The `ASSERT` makes the **link fail** if the heap would run into the stack limit. A memory corruption that would appear at run time, and be very hard to debug, becomes a clear error at build time.

> **A SMALLER RAM**
>
> With RAM `LENGTH = 8K`, `_estack` moves to `0x20002000` and `__StackLimit` to `0x20001000`. The sections `.data` + `.bss` + `.heap` still fit inside 8 KB, so there is no region overflow. However, the 4 KB heap now crosses `__StackLimit`, and the link fails because of the `ASSERT`.

2.2.3 Linker-Defined Symbols

Symbols such as `_sdata`, `_edata`, `_sidata`, `_sbss`, `_ebss` and `_estack` have **no storage**: no memory cell holds their value. The linker only assigns each one an **address**, and the address itself is the information.

```
extern uint32_t _sdata, _edata;   /* "there is something at this address" */
uint32_t *start = &_sdata;        /* correct: the address (0x20000000)    */
uint32_t  oops  = _sdata;         /* wrong: reads RAM at that address     */
```

They are declared as `extern` objects only so that C allows taking their address with `&`. Reading `_sdata` itself would read whatever happens to be in RAM at `0x20000000`. In assembly, `ldr r0, =_sdata` means `r0 = &_sdata;`. The boot code uses these addresses as loop bounds.

2.2.4 Orphan Sections

GCC lets the programmer put a variable or function in a section with any name:

```
int tagged __attribute__((section(".mysection"))) = 42;
```

If no rule in `SECTIONS` matches that name, the section is an **orphan**. The linker gives no error and places it by itself: an initialised writable section goes into RAM, right after `.data`, with its initial values in Flash; a read-only one goes into Flash.

> ⚠ **AN ORPHAN IS NOT INITIALISED BY THE STARTUP CODE**
>
> The startup code copies only the range `_sdata` … `_edata`. The orphan lies after `_edata`, outside that range, so its initial value is never copied: `tagged` reads 0 instead of 42. The link succeeds without warnings; the map file shows where the orphan ended up. Every section the program uses needs a rule in the linker script.

2.3 Section Placement of C Declarations

The section of each kind of declaration follows the mapping above. A few cases are easy to get wrong:

Case	Where it goes	Why
`int z = 0;`	`.bss`	"starts at zero" is exactly what `.bss` guarantees: no reason to store the zero in flash
`const char *p = "hi";`	`"hi"` → `.rodata` (FLASH), `p` → `.data` (RAM)	two distinct objects: the characters are constant, the pointer is an initialized writable variable
local variable (`int local = 7;` inside a function)	**no section**: the stack	it exists only while the function runs and has no fixed address
`static` local (`static int s = 3;` inside a function)	`.data` (or `.bss`)	it keeps one fixed address for the whole run, like a global; only its name is private

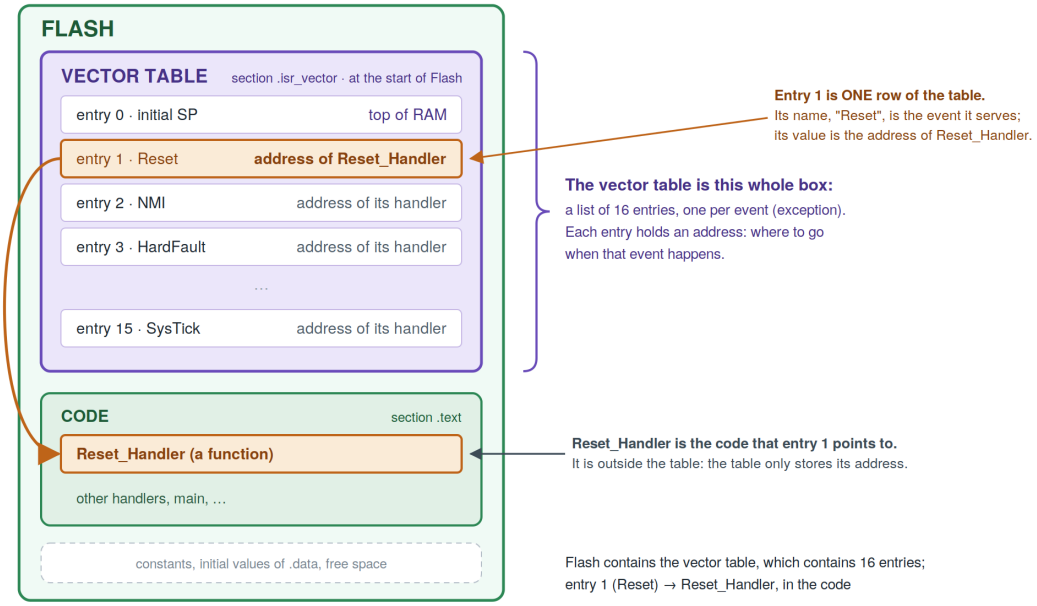
> ⚠ **`const` POINTER VS POINTER TO `const`**
>
> In `const char *greeting = "hi";` the `const` applies to the **characters**, not to the pointer. The pointer can still be changed (`greeting = "other";`), so it is an initialized writable variable and goes to `.data` (`D`), while `"hi"` goes to `.rodata`. Only `const char * const greeting = "hi";` makes the pointer itself constant, placing it in `.rodata` (`R`) as well. Reading rule: start from the name and read leftwards: "greeting is a pointer to constant chars".

3 The Vector Table

The **vector table** is a **table of addresses** at the **start of Flash**, one 32-bit word per entry. The CPU reads it by itself, without running any software: at reset, and every time an exception (an interrupt or a fault) occurs, to find the code it must run.

Entry	Address	Content
0	`0x00`	Initial value of the stack pointer (`_estack`)
1	`0x04`	Address of `Reset_Handler`, the first code to run
2–15	`0x08` – `0x3C`	Addresses of the handlers of the other exceptions (NMI, HardFault, SysTick, …)

Conceptual view: where the vector table is



At reset the CPU loads entry 0 into SP and entry 1 into PC, and starts executing `Reset_Handler`. Every address of code in the table is stored with the Thumb bit set: a handler at `0x40` is stored as `0x41`. The full list of exceptions is in the chapter on the ARM Cortex-M3 architecture.

The CPU looks for the table at address `0x0`, because `VTOR`, the register that holds its address, resets to `0x0000_0000`: this is why the linker script places `.isr_vector` first in FLASH. The program can later move the table, for example to a copy in RAM, by writing `VTOR`.

3.1 KEEP and Section Garbage Collection

With the option `-Wl,--gc-sections`, the linker performs **garbage collection of sections**:

1. it starts from the `ENTRY` symbol (`Reset_Handler`);
2. it follows every reference from one section to another (calls, pointers, addresses);
3. it **drops every input section that nothing references**.

This is useful to trim unused code, but it is dangerous for data that **only the hardware reads**. No code references the vector table: the CPU reads it by itself at reset. For the linker it is therefore unused, and it would be discarded.

`KEEP(*(.isr_vector))` marks those input sections as **always kept**, whatever the garbage collector decides.

`--gc-sections`	`KEEP`	Result
off	present or absent	nothing is collected: `KEEP` changes nothing, it only acts as insurance
on	present	`.isr_vector` is kept at `0x0`: the image works
on	absent	`.isr_vector` is discarded: the image does not boot

> ⚠ **WHAT HAPPENS WITHOUT `KEEP`**
>
> 1. `.isr_vector` shrinks to size 0; the map file lists it under *Discarded input sections*.
> 2. `.text` becomes the first section in FLASH, so `Reset_Handler` lands at `0x0`.
> 3. At reset the CPU reads the first two words at `0x0`, which are now **instructions**, and loads them as SP and PC.
> 4. With a meaningless SP and PC, execution faults immediately and the CPU ends in **lockup**.

4 The Startup Code: `boot/startup.s`

The **startup code** (`boot/startup.s`) is the **first code the CPU executes** after reset, before `main()`. It contains the **vector table** and `Reset_Handler`, the function that prepares the machine to run C code.

```
// THE COMPLETE STARTUP CODE


/* Assembler settings */
    .syntax unified             /* modern ARM syntax */
    .cpu cortex-m3              /* only M3 instructions */
    .thumb                      /* Thumb code only */


/* 1. Vector table */
    .section .isr_vector,"a",%progbits
    .global isr_vector
    .type isr_vector, %object
isr_vector:
    .word _estack               /* 0: initial SP */
    .word Reset_Handler         /* 1: reset */
    .word NMI_Handler           /* 2: NMI */
    .word HardFault_Handler     /* 3: HardFault */
    .word MemManage_Handler     /* 4: MemManage */
    .word BusFault_Handler      /* 5: BusFault */
    .word UsageFault_Handler    /* 6: UsageFault */
    .word 0                     /* 7: reserved */
    .word 0                     /* 8: reserved */
    .word 0                     /* 9: reserved */
    .word 0                     /* 10: reserved */
    .word SVC_Handler           /* 11: SVCall */
    .word DebugMon_Handler      /* 12: debug monitor */
    .word 0                     /* 13: reserved */
    .word PendSV_Handler        /* 14: PendSV */
    .word SysTick_Handler       /* 15: SysTick */
    .size isr_vector, . - isr_vector
```

```
/* Default handlers: weak aliases of Default_Handler */
    .weak      NMI_Handler
    .thumb_set NMI_Handler, Default_Handler
    .weak      HardFault_Handler
    .thumb_set HardFault_Handler, Default_Handler
    .weak      MemManage_Handler
    .thumb_set MemManage_Handler, Default_Handler
    .weak      BusFault_Handler
    .thumb_set BusFault_Handler, Default_Handler
    .weak      UsageFault_Handler
    .thumb_set UsageFault_Handler, Default_Handler
    .weak      SVC_Handler
    .thumb_set SVC_Handler, Default_Handler
    .weak      DebugMon_Handler
    .thumb_set DebugMon_Handler, Default_Handler
    .weak      PendSV_Handler
    .thumb_set PendSV_Handler, Default_Handler
    .weak      SysTick_Handler
    .thumb_set SysTick_Handler, Default_Handler

/* 2. Reset_Handler */
    .section .text,"ax",%progbits
    .global Reset_Handler
    .type Reset_Handler, %function
    .thumb_func
Reset_Handler:

    /* Step 1: mask interrupts */
    cpsid i                    /* PRIMASK = 1 */

    /* Step 2: set SP */
    ldr r0, =_estack
    mov sp, r0

    /* Step 3: copy .data FLASH -> RAM */
    ldr r0, =_sidata           /* src */
    ldr r1, =_sdata            /* dst */
    ldr r2, =_edata            /* end */
copy_data:
    cmp r1, r2                 /* while (dst < end) */
    bhs copy_data_done
    ldr r3, [r0], #4           /* r3 = *src++ */
    str r3, [r1], #4           /* *dst++ = r3 */
```

```
    b copy_data
copy_data_done:

    /* Step 4: zero .bss */
    ldr r1, =_sbss              /* dst */
    ldr r2, =_ebss              /* end */
    movs r3, #0
zero_bss:
    cmp r1, r2                  /* while (dst < end) */
    bhs zero_bss_done
    str r3, [r1], #4            /* *dst++ = 0 */
    b zero_bss
zero_bss_done:

    /* Step 5: VTOR = &isr_vector */
    ldr r0, =0xE000ED08
    ldr r1, =isr_vector
    str r1, [r0]

    /* Step 6: SHCSR |= bits 16-18 */
    ldr r0, =0xE000ED24
    ldr r1, [r0]
    orr r1, r1, #0x00070000
    str r1, [r0]

    /* Step 7: CCR |= DIV_0_TRP */
    ldr r0, =0xE000ED14
    ldr r1, [r0]
    orr r1, r1, #(1 << 4)
    str r1, [r0]

    /* Step 8: priorities (SHPR1-3) */
    ldr r0, =0xE000ED18
    movs r1, #0x00
    strb r1, [r0, #0]           /* MemManage */
    strb r1, [r0, #1]           /* BusFault */
    strb r1, [r0, #2]           /* UsageFault */
    movs r1, #0x80
    strb r1, [r0, #7]           /* SVCall */
    movs r1, #0xE0
    strb r1, [r0, #10]          /* PendSV */
    strb r1, [r0, #11]          /* SysTick */
```

```
    /* Step 9: unmask interrupts */
    cpsie i                     /* PRIMASK = 0 */

    /* Step 10: call main, then hang */
    bl main
hang:
    b hang

    .ltorg                      /* literal pool */
    .size Reset_Handler, . - Reset_Handler

/* 3. Default_Handler */
    .global Default_Handler
    .type Default_Handler, %function
    .thumb_func
Default_Handler:
    b Default_Handler           /* for (;;) {} */
    .size Default_Handler, . - Default_Handler
```

The order of the steps is dictated by **dependencies**: nothing may interrupt a half-built system, C needs a stack, globals must hold their values before any C code reads them, and **the configuration must be complete before interrupts are accepted**.

Step	Action	Reason
1	mask interrupts (`cpsid i`)	nothing may interrupt a half-built system
2	set SP	C functions need a stack
3	copy `.data` FLASH → RAM	initialized globals get their values
4	zero `.bss`	C guarantees uninitialized globals start at 0
5–8	configure the CPU	vector table address, fault handlers, divide-by-zero trap, priorities
9	unmask interrupts (`cpsie i`)	setup is complete
10	`bl main` , then loop forever	hand over to C; on bare metal there is nothing to return to

WHY ASSEMBLY

When the startup code begins, the stack has not been set up yet and `.data` and `.bss` are not initialized. C code takes all of these for granted, so it cannot be trusted until the startup code has finished. On a desktop this job is done by **crt0**, the C library's startup code; on bare metal it is excluded (`-nostartfiles`) and written by hand.

4.1 Assembler Directives

Lines starting with a **dot produce no instructions**; they **steer the assembler**.

Directive	Effect
`.section name,"flags"`	selects the section; `a` = occupies space in the image, `x` = executable
`.global name`	exports the name to other files and the linker (otherwise it behaves like a `static` C name)
`.thumb_func`	marks the next label as Thumb code, so its address is stored with bit 0 set
`.word X`	stores the 32-bit value X
`.type`, `.size`	tell `nm` and gdb whether a name is code or data, and its size
`.weak name`	declares a weak symbol, overridable by a normal definition
`.thumb_set a, b`	makes `a` an alias of `b`, marked as Thumb code
`.ltorg`	emits the literal pool here

5 Reset Handler: From Reset to main()

`Reset_Handler` takes the place of the operating system and of the C library: it brings the memory and the CPU into the state that C code assumes, and only then calls `main()`.

A STARTUP THAT ONLY CALLS `MAIN`

Consider a minimal `Reset_Handler` that executes only `bl main`, and a global `char data_message[] = "hello";`:

- the linker places the bytes `"hello"` in **FLASH** (LMA) and reserves space for `data_message` in **RAM** (VMA);
- nobody copies the bytes from FLASH to RAM;
- `main` reads `data_message` from **RAM**, where the copy was never written.

The result depends on what RAM contains at power-on. QEMU fills RAM with zeros, so the first byte is `'\0'` and the string reads as **empty**. Real hardware leaves random content, so the string reads as **garbage**. The program compiles and runs, but the data is wrong: this is the kind of silent error the copy loop prevents.

5.1 Weak Default Handlers

Every entry of the vector table must point to **valid code**, even for exceptions the program does not handle yet; reserved entries simply hold `0`. Instead of writing nine real handlers, the startup code uses two pieces:

- **`Default_Handler`**: a single catch-all function, an infinite loop (`for (;;) {}`);
- **weak names**: every handler name in the table is declared `.weak` and, by default, is an alias of `Default_Handler` (same address).

A **weak** symbol is a default that gives way: if another file defines a normal (strong) function with the same name, the linker uses that one and drops the weak alias. A handler is installed simply by writing a C function with the right name, without touching the assembly. The C equivalent of each weak name is:

```
void HardFault_Handler(void) __attribute__((weak, alias("Default_Handler")));
```

With only `UsageFault_Handler` defined in `main.c`:

Entry	Name	Defined in C?	CPU jumps to	`nm`
3	`HardFault_Handler`	no	`Default_Handler`	`W`, same address as `Default_Handler`
6	`UsageFault_Handler`	yes	the C function	`T`, its own address
15	`SysTick_Handler`	no	`Default_Handler`	`W`, same address as `Default_Handler`

WHY AN INFINITE LOOP

An unexpected exception is a bug. Freezing at a known place lets a debugger show immediately that PC is in `Default_Handler`, instead of letting the CPU run off into random memory.

5.2 Interrupt Masking and Stack Setup

`cpsid i` masks interrupts (PRIMASK = 1) until the vector table, handlers and priorities are in place, and `cpsie i` accepts them again; PRIMASK is described in the chapter on the ARM Cortex-M3 architecture.

SP is set explicitly even though the hardware already loaded it from entry 0 of the vector table, because `Reset_Handler` is not always reached through a real reset: a bootloader or debugger may jump to it with SP holding any value. SP is a special register, so the address is first loaded into `r0` with `ldr` and then moved with `mov sp, r0`.

FAILURE MODES OF A WRONG SP

With SP = `0x30000000` (outside RAM), the first push in `main` writes to a non-existent address and faults; reporting the fault requires stacking registers, which faults again, and the CPU enters lockup. With SP = `0x20000001`, the CPU silently clears the two low bits (SP is always word-aligned), so the stack starts at the bottom of RAM and grows *below* it, into an unmapped range where QEMU ignores writes: every call fails silently and nothing is printed.

5.3 Initializing .data and .bss

The `.data` copy translates `while (dst < &_edata) *dst++ = *src++;` with `r0` = `src` (`_sidata`), `r1` = `dst` (`_sdata`) and `r2` = end (`_edata`). It copies whole words, which is safe because the linker script aligns the start and end of `.data` to 4 bytes. Stepping through the loop in a debugger shows `r1` climbing `0x20000000`, `0x20000004`, … until it equals `&_edata`; only then does the string appear in RAM.

The `.bss` loop has the same shape without a source pointer: `while (dst < &_ebss) *dst++ = 0;`. When `.bss` is empty, `_sbss` equals `_ebss` and the loop exits immediately. Relying on the emulator zeroing RAM is incorrect: real SRAM is random at power-on. After these two loops, `Reset_Handler` provides exactly what a C runtime startup (crt0) provides.

5.4 Configuring the System Control Block

Steps 5–8 of `Reset_Handler` configure the System Control Block. The meaning of each register and field is described in the chapter on the ARM Cortex-M3 architecture; this section shows only what the boot code writes and why. All four registers are memory-mapped, so the startup code uses ordinary loads and stores: a plain store when the whole register is written, read-modify-write when other bits must keep their value, and byte stores for the priority bytes.

Step	Register	Operation	Effect
5	`VTOR` (`0xE000ED08`)	`= isr_vector`	exceptions use this table, wherever the image is placed
6	`SHCSR` (`0xE000ED24`)	`|= 0x00070000`	MemManage, BusFault, and UsageFault get their own handlers instead of escalating to HardFault
7	`CCR` (`0xE000ED14`)	`|= 1 << 4`	integer division by zero raises a UsageFault; `STKALIGN` stays 1, so `CCR` becomes `0x210`
8	`SHPR1` – `SHPR3` (`0xE000ED18` …)	byte stores	faults `0x00` , SVCall `0x80` , PendSV and SysTick `0xE0`

`VTOR` is written as a whole word, because all its bits are the table address. `SHCSR` and `CCR` contain status bits or bits that are already set (`STKALIGN`), so they are updated with read-modify-write and never overwritten. Each priority is a separate byte: viewing `SHPR1` – `SHPR3` as a 12-byte array at `0xE000ED18` , the boot code writes indices 0–2 (faults), 7 (SVCall), and 10–11 (PendSV, SysTick).

The effect of steps 6 and 7 is easy to observe with a division by a `volatile` zero. With both settings in place, a C function named `UsageFault_Handler` (which replaces the weak alias) runs and can report `CFSR = 0x2000000` , the `DIVBYZERO` bit. Without the `SHCSR` bits the fault escalates to HardFault and its kind is lost; without `DIV_0_TRP` the division silently returns 0 and `main` continues. The worked example and the fault status bits are in the fault status section of the Cortex-M3 chapter.

5.5 Additional Startup Work on Real Chips

QEMU's CPU runs at full speed out of reset. On a real microcontroller the startup code usually does more, often in a vendor function `SystemInit()` called from `Reset_Handler` :

- **Clock and PLL**: the chip starts on a slow internal oscillator, and a PLL multiplies the frequency up to full speed.
- **Flash wait states**: flash is slower than a fast CPU, so the flash controller must be told to insert wait cycles *before* the clock is raised, or the CPU reads garbage.
- **Watchdog**: a timer that resets the chip unless software periodically feeds it; on some chips it runs from reset, so boot code must feed or disable it.
- **FPU**: on a Cortex-M4F the floating-point unit is off at reset and must be enabled in **CPACR** (`0xE000ED88`), otherwise the first float instruction raises a UsageFault.
- **C++**: static constructors listed in `.init_array` must be called before `main` .

5.6 Verifying Boot State with gdb

Attached before the first instruction (see the toolchain chapter), gdb already shows SP = `0x20400000` and PC = `Reset_Handler`, while `x/2wx 0` shows `0x20400000 0x00000041`: no instruction has executed, so the hardware itself loaded both registers from the vector table, clearing the Thumb bit for PC. At `break main`, the SCB registers hold the values written in steps 5–8 (`VTOR` = `0x00000000`, `CCR` = `0x00000210`, `SHCSR` = `0x00070000`, `SHPR2` = `0x80000000`, `SHPR3` = `0xE0E00000`) and `info registers primask` reads 0 (it reads 0 at reset and 1 after `cpsid i`).

6 Freestanding C and Semihosting

6.1 Hosted and Freestanding Environments

The C standard defines two environments:

Aspect	Hosted	Freestanding
Example	desktop program	bare-metal Cortex-M3 image
Below the program	a C library (libc) and an OS	nothing
`printf`, `malloc`, `fopen`, `exit`	available	not available
Who starts `main()`	libc startup code, invoked by the OS	the application's own startup code

A freestanding environment still provides the headers that need **no run-time code**, only types, constants and macros supplied by the compiler:

Header	Provides
`stdint.h`	`uint32_t`, `int8_t`, …
`stddef.h`	`size_t`, `NULL`, `offsetof`
`stdbool.h`	`bool`, `true`, `false`
`stdarg.h`	`va_list` for variadic functions
`limits.h`, `float.h`	`INT_MAX`, `FLT_EPSILON`, …

There is no `stdio.h`, `stdlib.h` or `string.h`: no printing, no `malloc`, no `exit`, not even `memcpy`. This is why the boot code copies `.data` with its own loop.

6.2 The Layers Under printf

On a desktop, `printf` looks like a **single function** that writes to the screen. In reality it is the top of a stack of layers, and **most of the work happens in layers that a bare-metal system does not have**:

```
printf("x=%d\n", x)        program   formats the text into a buffer
  └─ write(1, buf, len)    libc      system call: the CPU enters the kernel
```

```
        └─ device driver    kernel    sends the bytes to the device
             └─ screen, UART, disk    hardware
```

`printf` itself only **formats**: it turns `"x=%d"` and `42` into the characters `x=42` in a buffer. **Getting those characters out of the machine** is the **OS's** job, reached through the `write()` system call. Many other libc functions have the same structure: a portable part on top, and a dependency on something underneath.

libc feature	Requires underneath
`printf`, `puts`	`write()`: a way to send bytes to a device
`malloc`, `free`	a heap region, grown with `sbrk()`
`exit`	something to return to
`main()` being called	crt0: sets up stack, `.data`, `.bss`, then calls `main`

Embedded C libraries such as **newlib** keep the portable part and leave the bottom functions (`write`, `sbrk`, `exit`, …) as **hooks** to implement for each board. The firmware described in these notes links **no libc at all**: there is nothing yet to put under the hooks, `Reset_Handler` already does the work of crt0, and the image contains no hidden code.

> **WHAT REPLACES `PRINTF`**
>
> Without libc, output is produced by the program itself: in this chapter through **semihosting**, and later through a UART driver and the kernel's own `kprintf` / `printf` (see the chapter on devices, interrupts, and system calls).

6.3 Why libgcc Is Kept

```
CFLAGS := -mthumb -mcpu=cortex-m3 -nostartfiles -ffreestanding -std=gnu17
LDFLAGS += -nostdlib -lgcc
```

The flags are described in the toolchain chapter. `libgcc` is kept because the compiler sometimes turns a plain C operator into a **call** to a helper when the CPU has no single instruction for it. These helpers need neither OS nor libc, and dropping them would make harmless code fail to link.

C code	CPU	GCC emits
`a / b` (32-bit)	Cortex-M0 (no divide instruction)	`bl __aeabi_idiv`
`a / b` (32-bit)	Cortex-M3	`sdiv` (hardware)
`x / y` (64-bit)	Cortex-M3	`bl __aeabi_uldivmod`

Calling `printf` in such a build always fails: with a toolchain that ships no C library, compilation stops with `stdio.h: No such file or directory`; with a toolchain that ships newlib, the header is found but the link fails with `undefined reference to 'printf'` because of `-nostdlib`.

6.4 Semihosting Output via bkpt 0xab

Without libc and without a UART driver, the program has no way of its own to print anything. All the output of the boot code is produced with **semihosting**: the program places `0x04` (`SYS_WRITE0`) in `r0` and the address of a NUL-terminated string in `r1`, then executes `bkpt 0xab`, and the host (QEMU or a debugger) prints the string on its terminal. The convention, the C wrapper `semihost_write0`, and its limits are described in the toolchain chapter.

> ⚠ **MULTIPLE INSTRUCTIONS IN ONE `__ASM__`**
>
> C concatenates adjacent string literals, so `"ldr r2, [r0]" "ldr r3, [r1]"` reaches the assembler as the single line `ldr r2, [r0]ldr r3, [r1]` and fails with `garbage following instruction`. Each instruction must end with `\n` (commonly `\n\t`) inside the string.